

INFORME TÉCNICO INTEGRADO

Adquisición, Procesamiento Relacional y Análisis Estadístico de Datos en Python

Un estudio detallado sobre la integración de múltiples orígenes de datos (ficheros planos, relacionales y jerárquicos) bajo el ecosistema de librerías científicas de Python y el posterior diagnóstico avanzado de calidad e integridad corporativa.

1. Introducción y Marco Teórico

En este marco inicial, nos introducimos exhaustivamente en los principios y mecánicas del manejo de datos. Aprendemos a utilizar las principales librerías del ecosistema Python, tales como `pandas`, `numpy`, y conectores especializados como `sqlite3` o `sqlalchemy`, con el propósito fundamental de adquirir, guardar y manipular datos de forma robusta y eficiente.

A lo largo del desarrollo del proyecto, se aborda la interconexión técnica con diferentes formatos de almacenamiento del mundo real: ficheros de texto estructurado de gran escala (CSV), hojas de cálculo comerciales (Excel), así como estructuras jerárquicas y anidadas complejas en notación de objetos (JSON). Paralelamente, se simulan las operaciones típicas de interacción con bases de datos relacionales directamente desde código estructurado, realizando consultas, altas y modificaciones mediante sentencias SQL embebidas.

2. Objetivos del Proyecto

El diseño metodológico del proyecto se orientó hacia la consecución de una serie de hitos formativos y operativos cruciales para cualquier entorno de analítica de datos:

- **Dominio del Ecosistema Científico:** Conocer en profundidad y explotar la sintaxis de las librerías fundamentales para ciencia de datos en Python.
- **Adquisición Heterogénea:** Leer datos provenientes de múltiples canales y formatos de forma estructurada e integrada.
- **Persistencia y Serialización:** Guardar y exportar subconjuntos analíticos estructurados a formatos estandarizados de uso común.
- **Interacción Relacional Avanzada:** Conectarse eficientemente a motores de bases de datos relacionales, realizando operaciones CRUD (Consulta, Inserción, Actualización y Eliminación).
- **Robustez y Control de Errores:** Detectar y mitigar anomalías comunes asociadas al proceso crítico de ingesta y adquisición de datos.

- **Verificación y Validación de Calidad:** Comprobar la integridad física y conceptual básica de los conjuntos leídos para asegurar su validez analítica.

3. Procedimiento Empleado y Arquitectura del Script

La implementación técnica se estructuró a través de un pipeline secuencial automatizado en Python que consta de fases bien definidas, garantizando un flujo limpio desde el almacenamiento en disco hasta el análisis final.

Fase A: Carga y Unión Dinámica (Apartado 0)

El script utiliza un mecanismo dinámico para detectar la ubicación del entorno de ejecución mediante la variable de sistema `os.path.abspath` o el directorio de trabajo activo. A partir de allí, se realiza la lectura concurrente de tres fuentes de datos críticas:

1. **Pedidos de Venta:** Almacenados en un archivo plano `pedidos.csv`, utilizando como delimitador el carácter barra vertical (`|`) y codificación de caracteres UTF-16. Posterior a su carga, se realiza un saneamiento estricto de las etiquetas de columna mediante `.str.strip()`.
2. **Catálogo de Productos:** Alojado en una base de datos relacional local `productos.db`. Se utiliza el conector nativo `sqlite3` para inyectar la consulta declarativa `SELECT * FROM productos`, mapeándola instantáneamente a un DataFrame estructurado.
3. **Registro Demográfico de Clientes:** Ingerido desde un fichero jerárquico anidado `clientes.json`. Para solucionar la complejidad de las subestructuras de direcciones, se invoca el método avanzado `pd.json_normalize()`, el cual aplanar los nodos internos como `direccion.pais` y `direccion.ciudad` hacia columnas vectoriales planas de primer nivel.

Nota de Robustez Industrial: Previo a realizar las uniones (joins), el procedimiento obliga a una homologación estricta de tipos de datos. Las columnas clave `id_cliente` y `id_producto` son convertidas explícitamente a tipo cadena limpia (`.astype(str).str.strip()`). Esto anula cualquier fallo por discrepancias de formato numérico o caracteres ocultos invisibles, logrando cruzar un total final de 59,143 registros.

Fase B: Diagnóstico de Tipos e Inspección de Valores Nulos (Apartados 1 y 2)

Una vez obtenida la base unificada, se aplican los métodos analíticos de diagnóstico estructural: `.shape`, `.columns`, `.dtypes`, e `.info()`. Posteriormente, se cuantifican las ausencias de información sumando y calculando los porcentajes exactos de valores NaN fila por fila y columna por columna.

Fase C: Normalización Numérica y Estadística Descriptiva (Apartado 3)

Para habilitar la computación matemática, los atributos financieros y lógicos del negocio (cantidad, precio_unitario, total, precio_base, descuento y stock) se fuerzan al tipo de dato continuo `float64` recurriendo a `pd.to_numeric(..., errors="coerce")`. Con ello, se calculan las medidas de tendencia

central (Media, Mediana, Moda), métricas de dispersión (Desviación Estándar, Varianza, Rangos) y recuentos de cardinalidad única para variables cualitativas.

4. Análisis Detallado de Resultados y Diagnóstico de Integridad

La ejecución arrojó métricas críticas y reveladoras respecto al estado real del repositorio corporativo. A continuación, se detallan los hallazgos analíticos extraídos de los resúmenes impresos en consola:

4.1 Análisis del Cruce y la Desconexión Relacional de Clientes

El dataset final consta de un volumen masivo de 59,143 filas y 25 columnas. No obstante, al auditar la calidad mediante la matriz de nulos (Apartado 2.1 y 2.4), se detectó una anomalía severa en los datos. El 100% de las columnas demográficas añadidas desde el archivo JSON de clientes (nombre, email, telefono, direccion, genero, edad y nivel_fidelizacion) presentan un valor nulo absoluto (100.00% de valores faltantes).

Diagnóstico Crítico del Error: El cruce de datos (Left Join) falló estructuralmente debido a que los valores contenidos en la columna de enlace `id_cliente` dentro del archivo de `pedidos.csv` y el archivo `clientes.json` no poseen coincidencia ni correspondencia real alguna en sus rangos lógicos de identificación. Esto causó que 0 de los 40,000 pedidos originales lograran ligarse con un registro de cliente real, arrojando estadísticas vacías en la dimensión demográfica.

4.2 Comportamiento de Métricas Financieras y Anomalías de Entrada

El análisis descriptivo de las variables cuantitativas desveló un comportamiento distorsionado por la presencia de códigos de error numéricos inyectados como datos reales de negocio:

Métrica Estadística	Columna 'cantidad'	Columna 'precio_unitario'	Columna 'total'
Conteo de Registros Validados	58,233.00	58,152.00	58,152.00
Media Calculada	-1,660.36	-714.41	3,493.75
Mediana Estadísticas	5.00	954.06	3,876.27
Valor Mínimo Registrado	-99,999.00	-99,999.00	-99,999.00
Valor Máximo Registrado	10.00	2.000.00	19,994.40

Como puede apreciarse en la tabla, el valor mínimo en múltiples columnas estratégicas se sitúa exactamente en -99,999.00. Esto obedece a una mala práctica extendida en la captura de registros origen, donde se introduce dicho número para denotar un dato no ingresado o un error de sistema.

La presencia de este valor sesga por completo las medias aritméticas (provocando, por ejemplo, una media irreal de -1,660.36 unidades por compra en la empresa). Sin embargo, al observar la mediana (5.00 unidades), comprobamos que el comportamiento central del negocio se mantiene estable una vez ignorado el ruido. Se detectaron de forma exacta 970 valores negativos espurios en la columna cantidad. Por otra parte, la columna total reporta una volatilidad inmensa con una varianza acumulada superior a los 213 millones, producto del mismo fenómeno de contaminación de datos.

4.3 Hallazgos Adicionales de Calidad en Cadenas y Categorías

Al auditar las columnas cualitativas (Apartado 3.6), saltaron a la luz múltiples defectos de normalización léxica en los campos clave operacionales:

- **Errores Explícitos de Extracción:** Cadenas de texto como "###ERROR###" aparecen de forma reiterada y masiva. Ocupan el rango número uno de presencia en variables como País envío (928 repeticiones) y proveedor (1,183 repeticiones), y se sitúan en segundo lugar en fecha_pedido (1,008 repeticiones) y nombre_producto (1,867 repeticiones).
- **Truncamiento y Variabilidad de Escritura:** En el método de pago y estado de pedido conviven formas íntegras y abreviaturas rotas (por ejemplo: "Tarjeta" con 223 filas en "Ta", o "Pendiente" conviviendo con 291 instancias de "Pe"). El script solventa analíticamente esto mediante máscaras booleanas complejas usando `.str.lower().isin()` para homogeneizar la lógica comercial.
- **Falta de Fechas Válidas:** La columna fecha_pedido aloja enteros truncados como 20 (1,014 registros) o caracteres nulos puros (##), imposibilitando un análisis temporal lineal sin un proceso previo de parseo riguroso.

5. Conclusiones y Plan de Acción Propuesto

El desarrollo de la práctica ha cumplido cabalmente con los objetivos fijados para la especialización en Ciencia de Datos. Ha demostrado que el dominio de las librerías técnicas es condición necesaria, pero no suficiente, si no va acompañado de un análisis crítico respecto a la veracidad, integridad y limpieza profunda de las fuentes de datos.

Recomendación Ingeniería de Datos: Se aconseja rechazar el origen de los ficheros de producción e implementar una fase de Data Cleansing previa a cualquier modelado descriptivo o predictivo. Dicha fase deberá sustituir los códigos placeholders (-99,999) por vacíos verdaderos (`np.nan`), mapear las abreviaturas duplicadas de estados mediante diccionarios explícitos, y renegociar la extracción del JSON de clientes bajo una clave primaria compatible con el histórico de ventas.

Fin del Informe Técnico Integrado